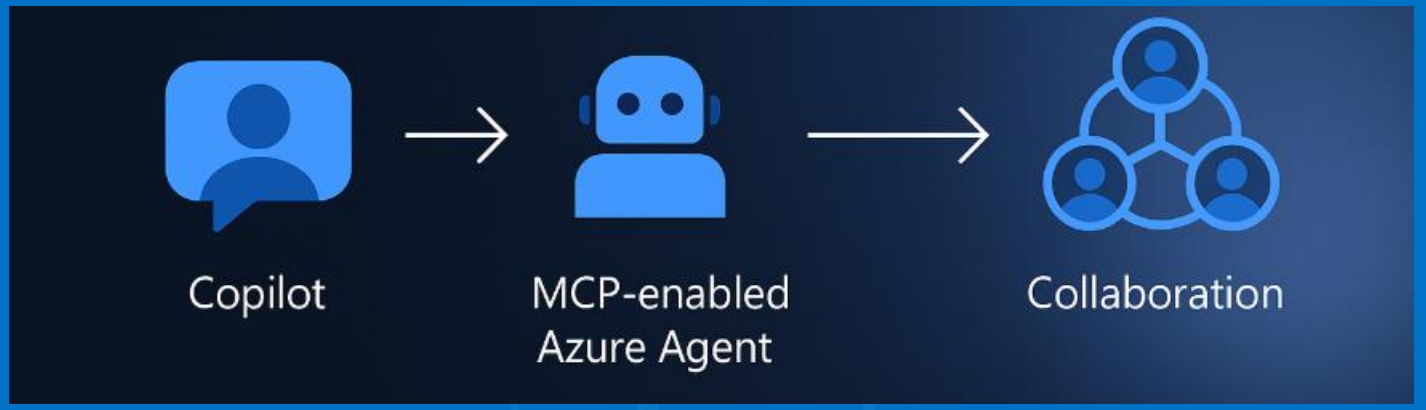
Beyond Copilot: Architecting MCP-Enabled Azure Agents



Hemamalini Nithyanandam

Agenda

01

Evolution from Copilots to Collaborative Agents

Understanding the shift from simple copilots to autonomous, multi-agent systems.

02

Model Context Protocol (MCP) and Tool Integration

Deep dive into MCP and how to integrate various tools for enhanced agent capabilities.

03

AI Agent Frameworks

Exploring popular frameworks like LangGraph, CrewAI, Semantic Kernel, and AutoGen.

04

Microsoft Agent Framework Architecture and Components

A detailed look at Microsoft's approach to agent architecture and its core components.

05

Azure AI Foundry Integration and Deployment

Leveraging Azure AI Foundry for seamless integration and deployment of AI agents.

06

Hands-on Demo: Building a Pirate Joke Agent using Microsoft Agent Framework

A practical demonstration of creating an interactive AI agent.

Who Am I ….

Professional

- Cloud Engineering Expert in HP
- 13+ years IT experience
- AWS Solutions Architect
- Data Engineer
  &
- AI Enthusiast

Personal

- Punnagai Foundation
- Certified Yoga Trainer
- Blogger

https://www.linkedin.com/in/hemamalini-nithyanandam/

From Copilot to Collaboration — Building the Future of Autonomous AI on Azure.

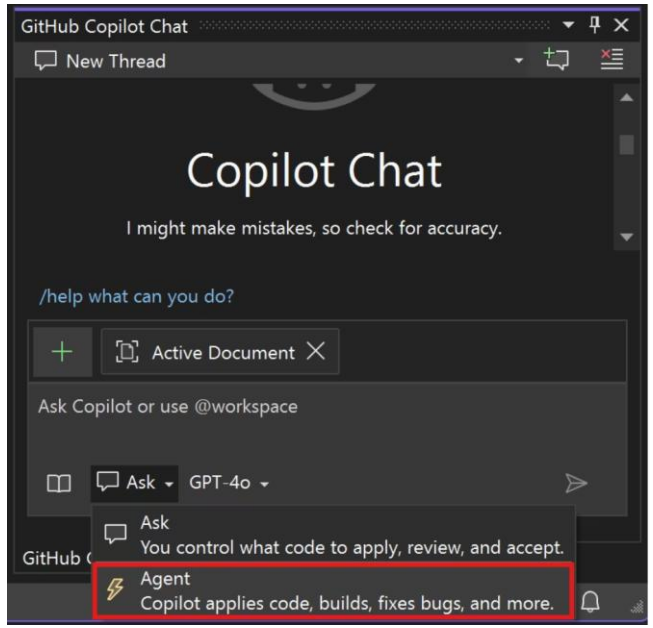
AI Assistants Changed Everything — But Not Enough

> Two years ago, Copilot rewired how we code, write, and build. For the first time, AI sat *beside us* — not *above us*.

But here's the catch: copilots still wait for us to tell them what to do. They don't **plan**, they don't **negotiate**, they don't **act together**.

The current generation of AI assistants operates in a fundamentally reactive mode. They excel at augmenting human decision-making but lack the autonomous reasoning, multi-agent coordination, and proactive planning capabilities required for truly intelligent systems.

Imagine if they could. That's exactly what MCP-enabled agent architectures unlock — a future where AI systems collaborate autonomously, reason about complex tasks, and coordinate actions across distributed environments.



The Evolution of Microsoft's AI Platform

Understanding how we arrived at today's agent-centric architecture requires tracing the rapid evolution of Microsoft's AI platform — from early coding assistants to sophisticated multi-agent frameworks.

1. **Jun 2021: GitHub Copilot Announced**
   Technical preview launched, introducing AI-assisted coding to developers worldwide
2. **Jun 2022: GitHub Copilot GA**
   General availability of basic coding assistant capabilities, establishing the copilot paradigm
3. **Mar 2023: Microsoft 365 Copilot + Semantic Kernel**
   M365 Copilot announced for Word, Excel, PowerPoint, Outlook, Teams; Semantic Kernel public repo launched for LLM orchestration
4. **May 2023: Windows Copilot Preview**
   OS-level integration announced at Build 2023, expanding copilot reach
5. **Sep 2023: AutoGen Released + Copilot Branding**
   Multi-agent framework introduced; Bing Chat rebranded to Microsoft Copilot; M365 Copilot GA announced
6. **Nov 2024: Anthropic Introduces MCP**
   Model Context Protocol launched as open standard for agent-tool communication
7. **Jan 2025: AutoGen v0.4 Major Redesign**
   Complete architectural overhaul focusing on scalability and enterprise patterns
8. **Mar 2025: MCP in Copilot Studio**
   Public preview brings standardized tool integration to low-code platform
9. **May–Aug 2025: MCP Across Developer Surfaces**
   Teams AI library GA with MCP support; Azure AI Foundry Agents add MCP tooling
10. **Oct 2025: Microsoft Agent Framework**
    Public preview announced — converging AutoGen and Semantic Kernel into unified SDK

MCP — The Language That Lets Agents Talk to Tools

Model Context Protocol (MCP) is an open standard that lets AI agents discover, connect, and collaborate with external tools and other agents — securely and consistently.

Why It Matters

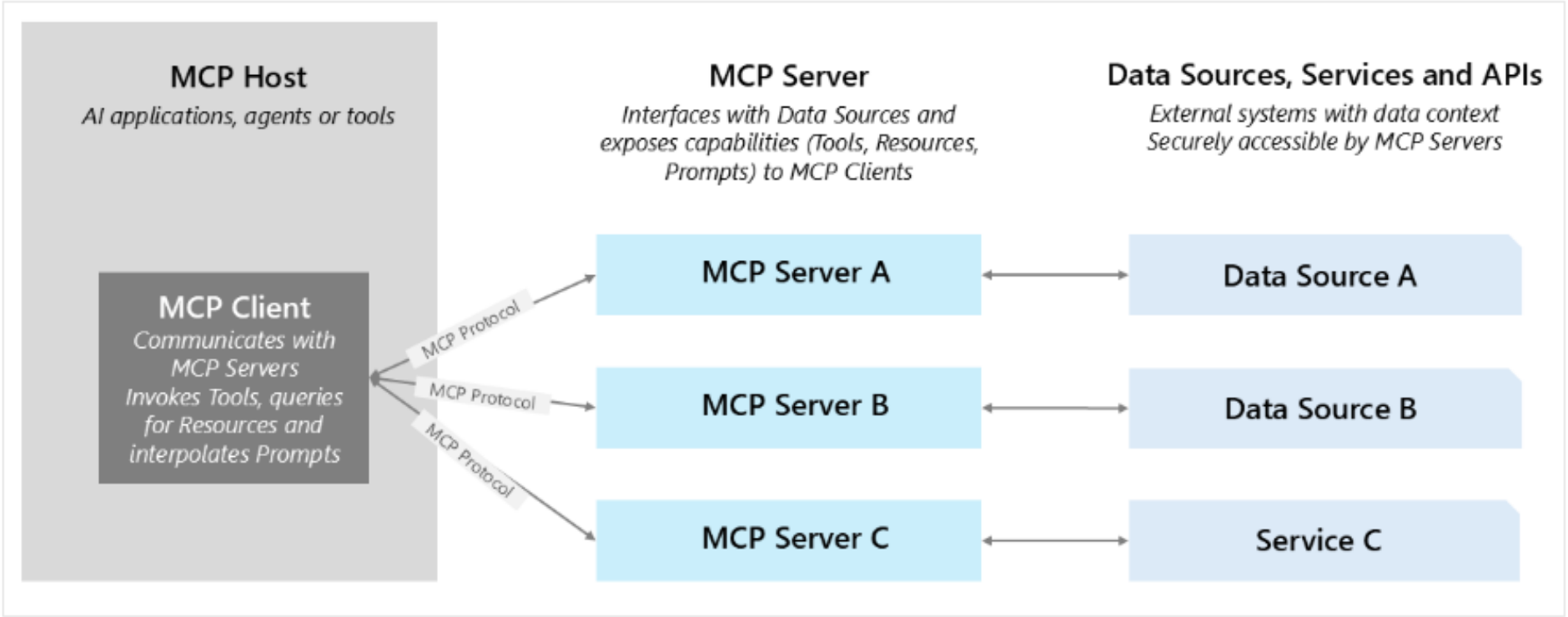
Modern AI systems are moving from single-task copilots to **connected, multi-agent ecosystems**. Each agent must discover, call, and reason over external tools, APIs, and knowledge sources. MCP standardizes that interaction — creating a universal interface for intelligent collaboration.

Dynamic Tool Discovery

MCP enables agents to **automatically identify and connect** to available external tools at runtime — eliminating hardcoded integrations and enabling truly adaptive intelligence. Agents can query capabilities, negotiate protocols, and establish secure connections without manual configuration.

Technical Foundation: MCP operates as a JSON-RPC 2.0 protocol with standardized schemas for tool registration, capability negotiation, and secure invocation — enabling interoperability across heterogeneous agent frameworks and runtime environments.

MCP Architecture



https://learn.microsoft.com/en-us/azure/api-management/mcp-server-overview

MCP Servers in Visual Studio / GitHub Copilot Agent Mode

What Are MCP Servers?

MCP servers expose **tool capabilities** (APIs, operations) to AI agents operating in environments like GitHub Copilot's agent mode. They allow agents to invoke external operations — file I/O, HTTP calls, GitHub actions, database queries, cloud service operations — via a unified protocol.

Local Execution

MCP servers can run **locally** on developer machines using stdio/command-based communication, providing low-latency access to file systems, local APIs, and development tools.

Remote Execution

Servers can also operate **remotely** via HTTP/SSE protocols, enabling enterprise-scale deployments with centralized governance, monitoring, and security controls.

Unified Interface

Regardless of deployment model, agents interact with MCP servers through the same standardized protocol, ensuring portability and consistency across environments.

MCP Servers in Visual Studio: Configuration & Security

How Visual Studio Connects

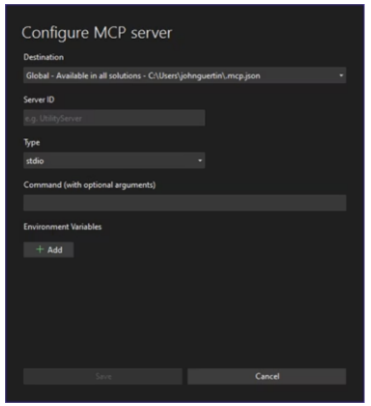
- Visual Studio (acting as MCP client) loads servers from `mcp.json` or `.mcp.json` configuration files in well-known locations (global, user, or solution-specific)
- Supports both **local (stdio/command)** and **remote (HTTP/SSE)** server types with automatic discovery
- On configuration save, VS **hot-reloads** and re-discovers tools dynamically, enabling real-time updates without IDE restart
- Tool metadata includes schemas, parameter validation, and execution context requirements

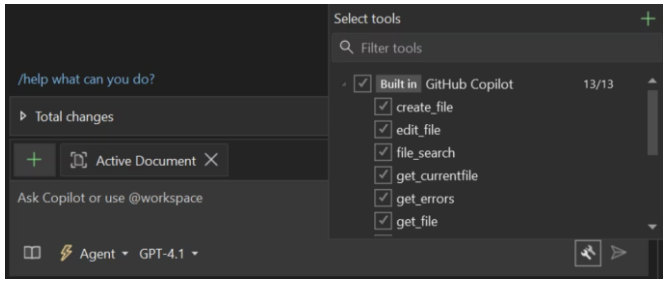
Security & Control

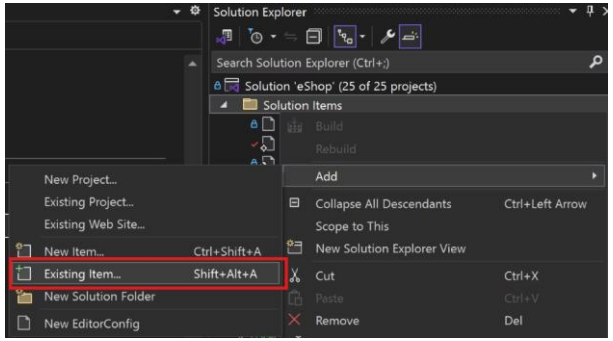
- All tools are **permission-gated** — users must explicitly approve tools before first invocation, establishing trust boundaries
- Enterprise administrators can **govern Agent Mode & MCP** via centralized Copilot policy, controlling tool availability, execution permissions, and audit logging
- Fine-grained access controls support role-based permissions and conditional access policies

https://learn.microsoft.com/en-us/visualstudio/ide/mcp-servers?view=vs-2022

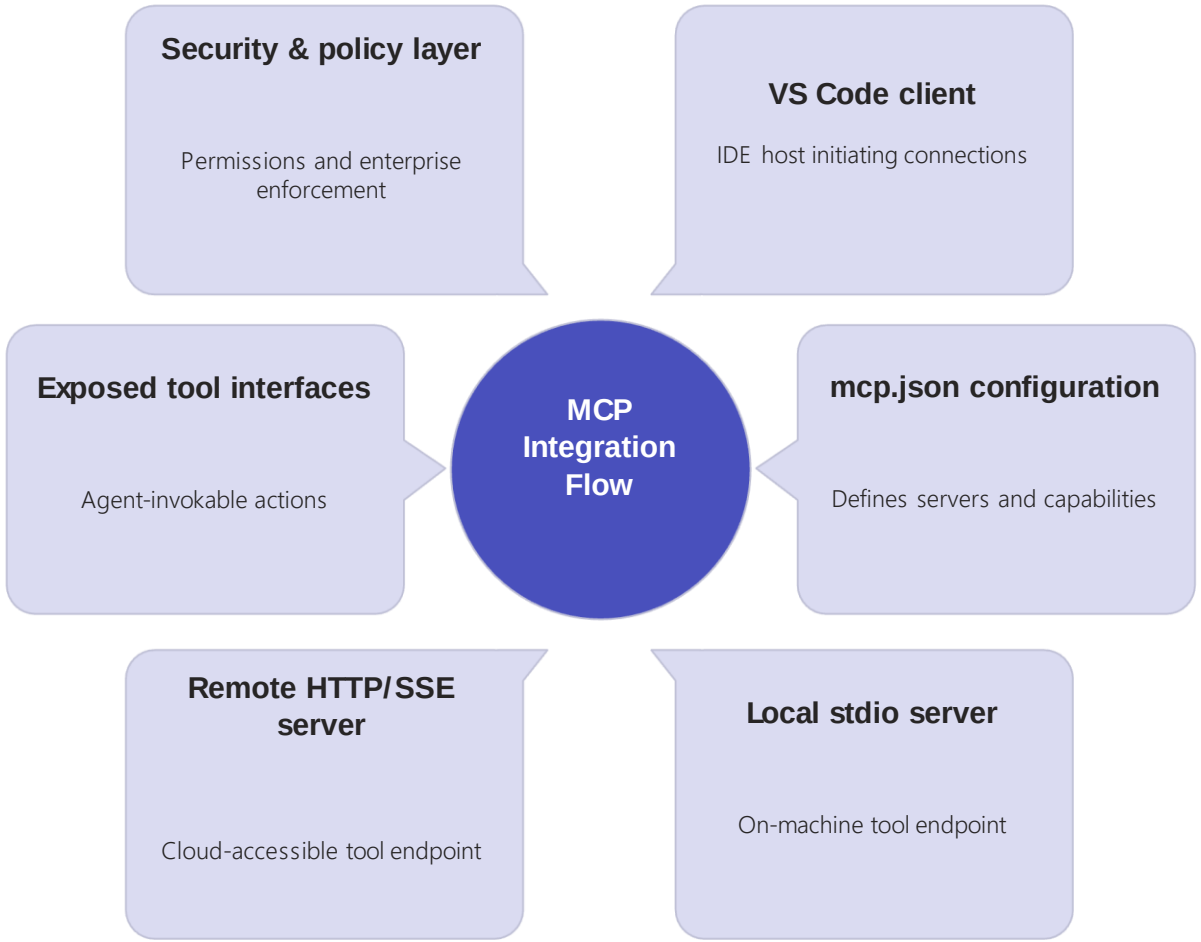
https://github.com/mcp?utm_source=vscode-website&utm_campaign=mcp-registry-server-launch-2025







MCP Integration Architecture



The MCP architecture in Visual Studio creates a robust, extensible foundation for agent-tool integration. Configuration files define server endpoints and capabilities, while the runtime manages discovery, connection lifecycle, permission enforcement, and error handling. This separation of concerns enables both rapid local development and secure enterprise deployments.

Building an MCP-Enabled GitHub Agent in VS Code

Enable **Copilot Chat** to perform real GitHub operations — branch creation, commits, pull requests — via a custom MCP server written in Python. This demonstrates how MCP transforms Copilot from a passive assistant into an executable DevOps agent.

> "MCP turns Copilot into an executable DevOps agent — no SDKs, no plugins, just JSON-RPC"

Component	Description
.mcp.json	Registers the server in VS Code with command: `python github_mcp_server.py`
github_mcp_server.py	Python MCP server exposing structured tools via JSON-RPC 2.0 protocol
Tools Exposed	`create_branch`, `commit_file`, `create_pull_request`, `list_issues`, `comment_issue`, `health`
Authentication	Fine-grained GitHub Personal Access Token in .env with Contents RW and PR RW permissions
Protocol Support	Implements `initialize` handshake and `logging/setLevel` for VS Code integration

This architecture enables natural language commands like "Create a feature branch and commit the updated README" to execute as actual GitHub API calls, with full audit trail and error handling.

Top AI Agent Frameworks

AI Agent Frameworks simplify the creation of intelligent, goal-driven systems by providing ready-made components for reasoning, memory, environment interaction, and tool integration. Understanding the landscape helps architects choose the right foundation for their use cases.

Agent Architecture

Defines decision-making models, memory structures, and action execution patterns — the cognitive foundation of intelligent behavior

Communication Protocols

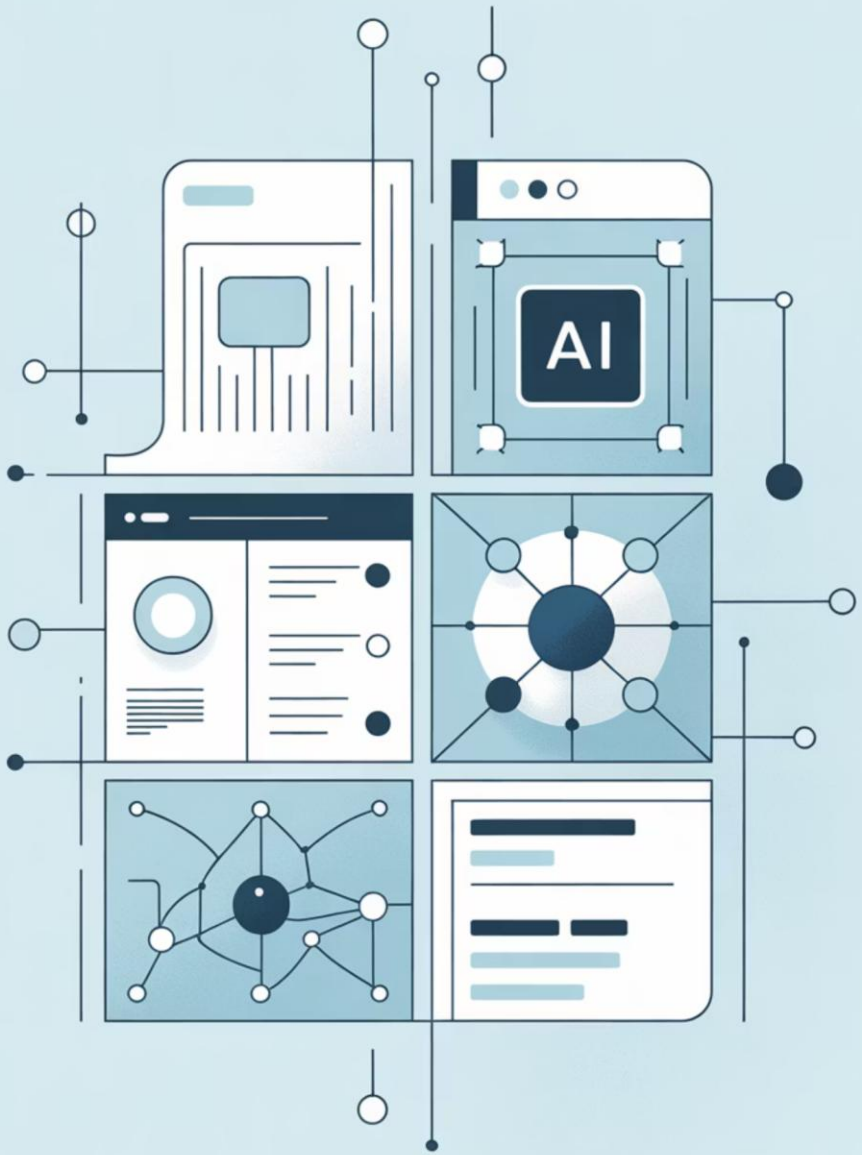
Enables agent-to-agent and agent-to-human collaboration through standardized message formats and conversation patterns

Integration Tools

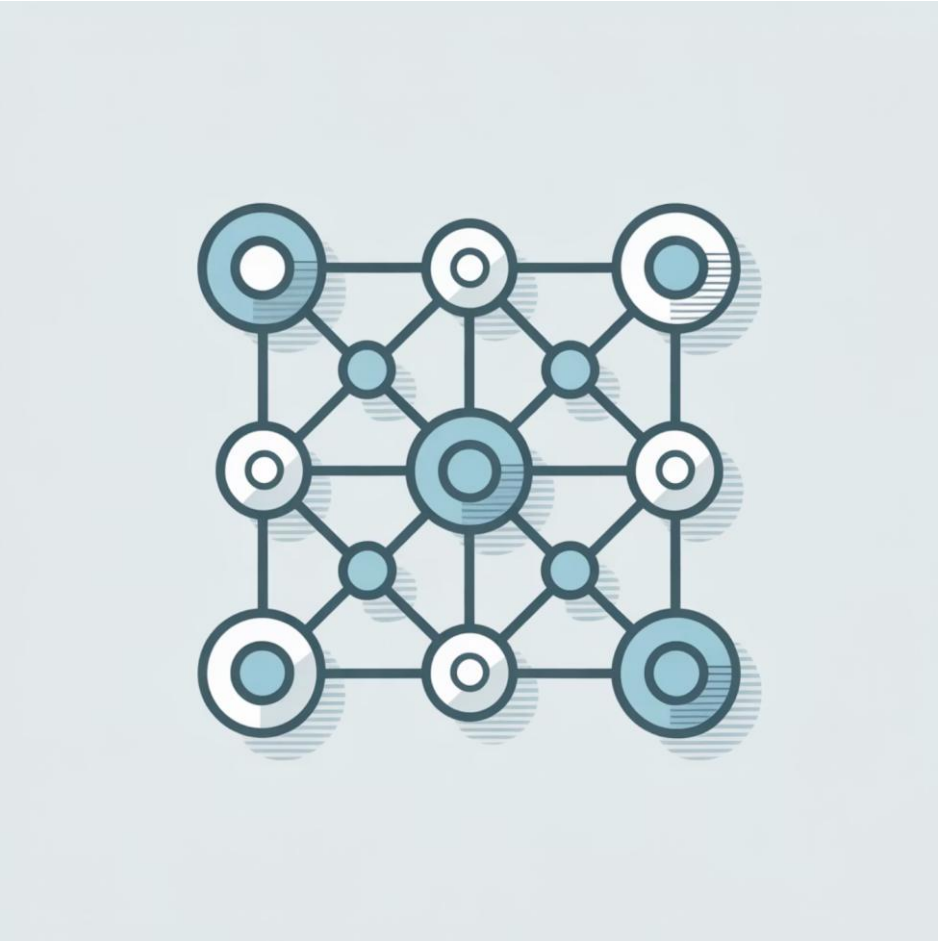
Connects agents to external APIs, data sources, MCP servers, and cloud services through unified interfaces

Learning Mechanisms

Supports continuous adaptation and task improvement through feedback loops, reinforcement learning, and context retention



LangGraph: Stateful Multi-Agent Orchestration



Purpose

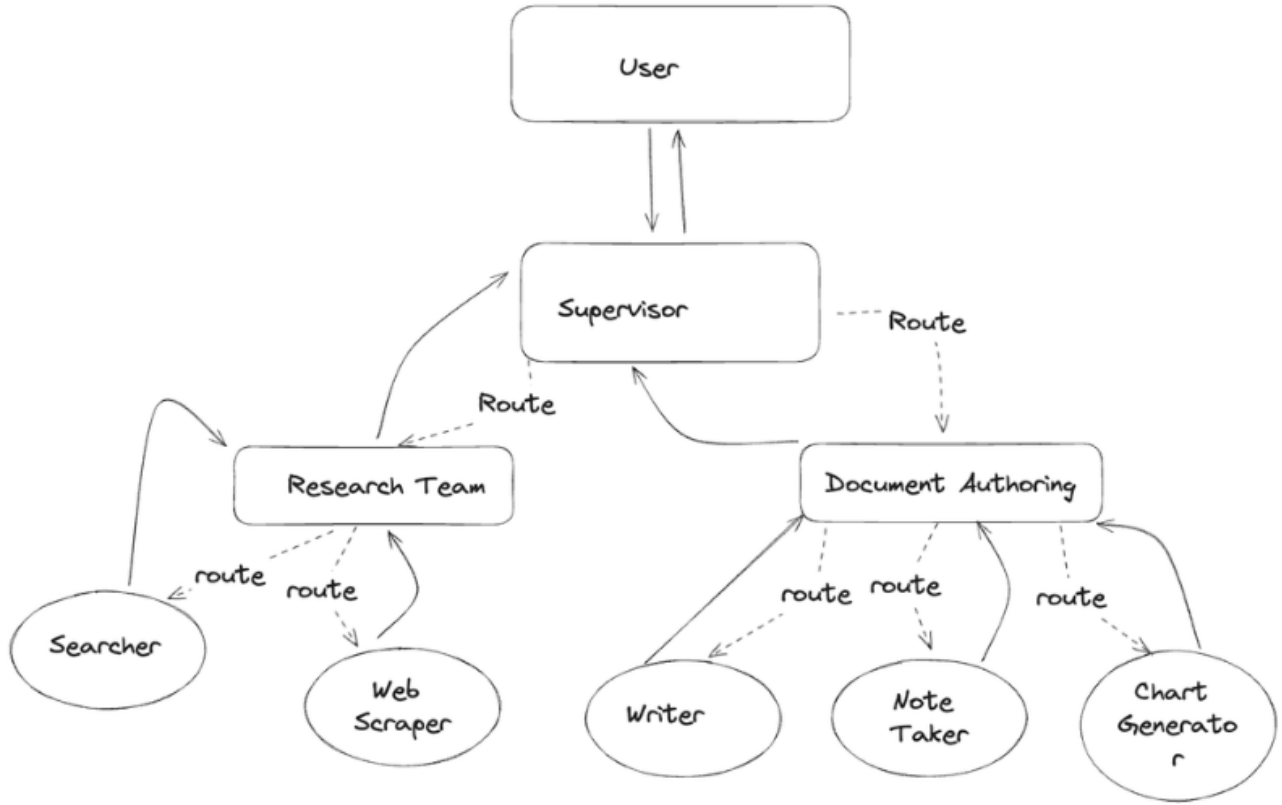
Framework built atop LangChain for multi-agent and stateful graph orchestration, enabling sophisticated workflow coordination.

Key Features

- Defines agent workflows as **graphs of nodes and edges**, where nodes represent agent states and edges represent transitions
- Manages **parallel, conditional, and cyclic** agent interactions with sophisticated control flow
- Handles context flow and event streaming across distributed agent systems
- Supports state persistence, checkpointing, and workflow resumption
- Provides visual workflow editors and debugging tools

Use Case

Coordinating multiple specialized agents for RAG pipelines, multi-step planning, evaluation workflows, or complex decision trees requiring state management and conditional branching.



https://blog.langchain.com/langgraph-multi-agent-workflows/

CrewAI: Team-Based Agent Collaboration

Purpose

Enables **team-based AI collaboration** where agents operate as coordinated roles, mimicking human team structures and workflows.

Role-Driven Design

Agents assume specific roles (Researcher, Writer, Reviewer, Quality Assurance) with defined responsibilities, expertise areas, and collaboration patterns.

Shared Memory

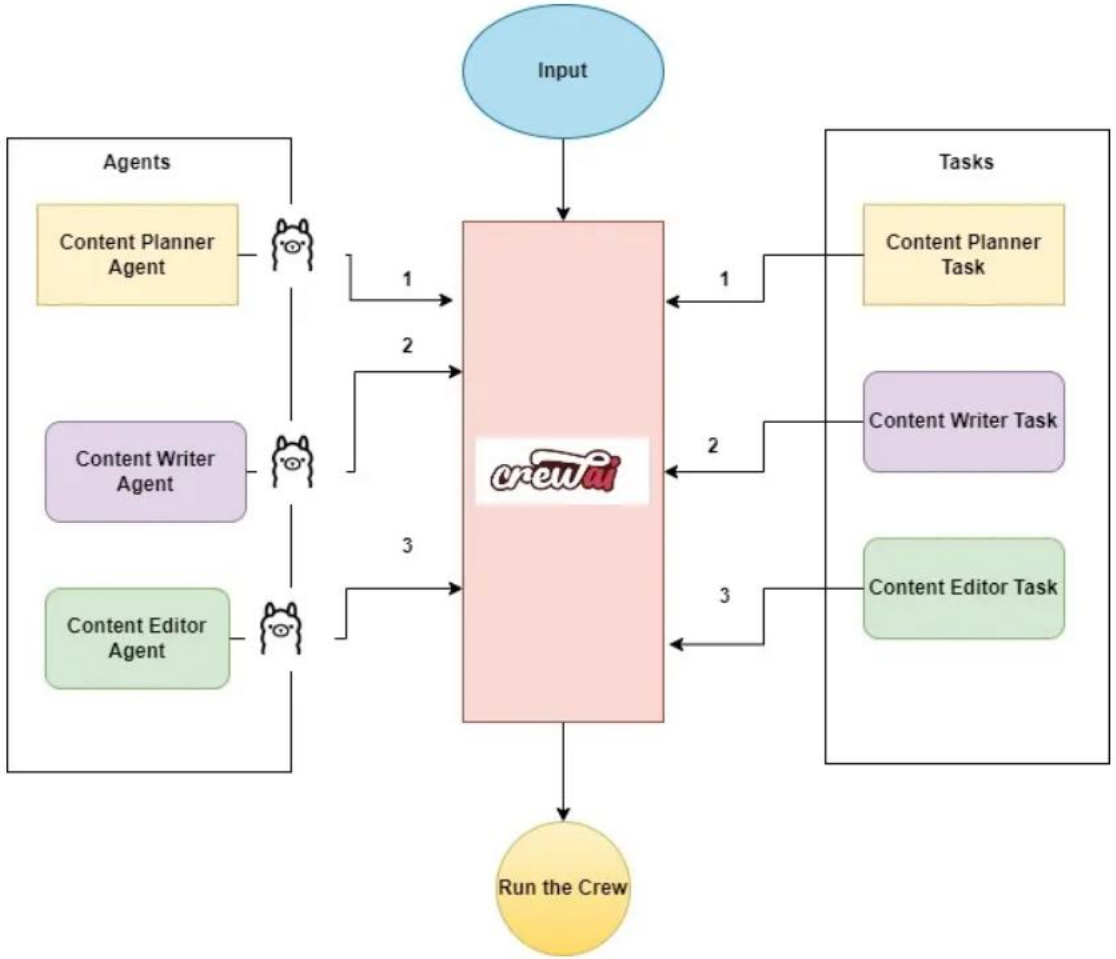
Agents maintain shared context for task coordination, enabling seamless handoffs and collaborative problem-solving across role boundaries.

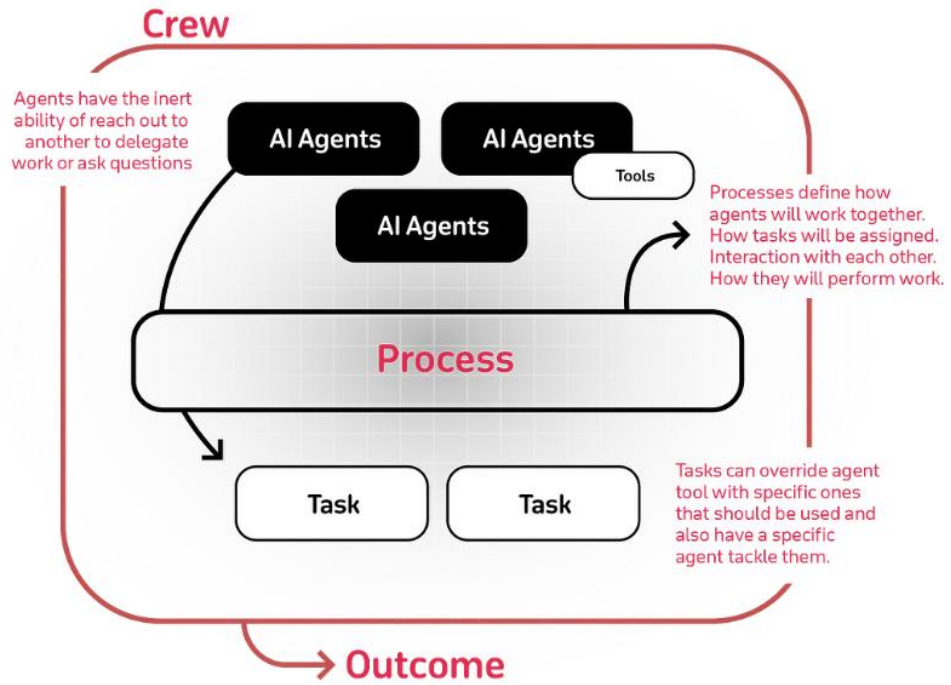
Planning Loop

Built-in communication and planning mechanisms coordinate multi-agent workflows, with automatic task decomposition and delegation.

Use Case

Automating research and content generation workflows with multiple reasoning agents — for example, a Researcher gathers information, a Writer creates content, and a Reviewer ensures quality and accuracy.





https://github.com/joaomdmoura/crewAI?ref=blog.composio.dev

https://medium.com/the-ai-forum/create-a-blog-writer-multi-agent-system-using-crewai-and-ollama-f47654a5e1cd

Semantic Kernel: Enterprise LLM Integration

Purpose

SDK for integrating LLMs, memory, and skills into enterprise applications with strong Azure ecosystem integration.

Key Features

- "Skills" abstraction for reusable functions or APIs — modular capabilities that any agent can invoke
- Memory Store for long-term context and embeddings, enabling semantic recall across sessions
- Planner for task decomposition and orchestration, breaking complex goals into executable steps
- Native connectors to Azure OpenAI, Copilot, and external APIs
- Support for multiple programming languages (.NET, Python, Java)

Use Case

Build context-aware, orchestrated agents inside Microsoft ecosystem — ideal for enterprise applications requiring Azure AD integration, compliance controls, and cloud-native scaling.

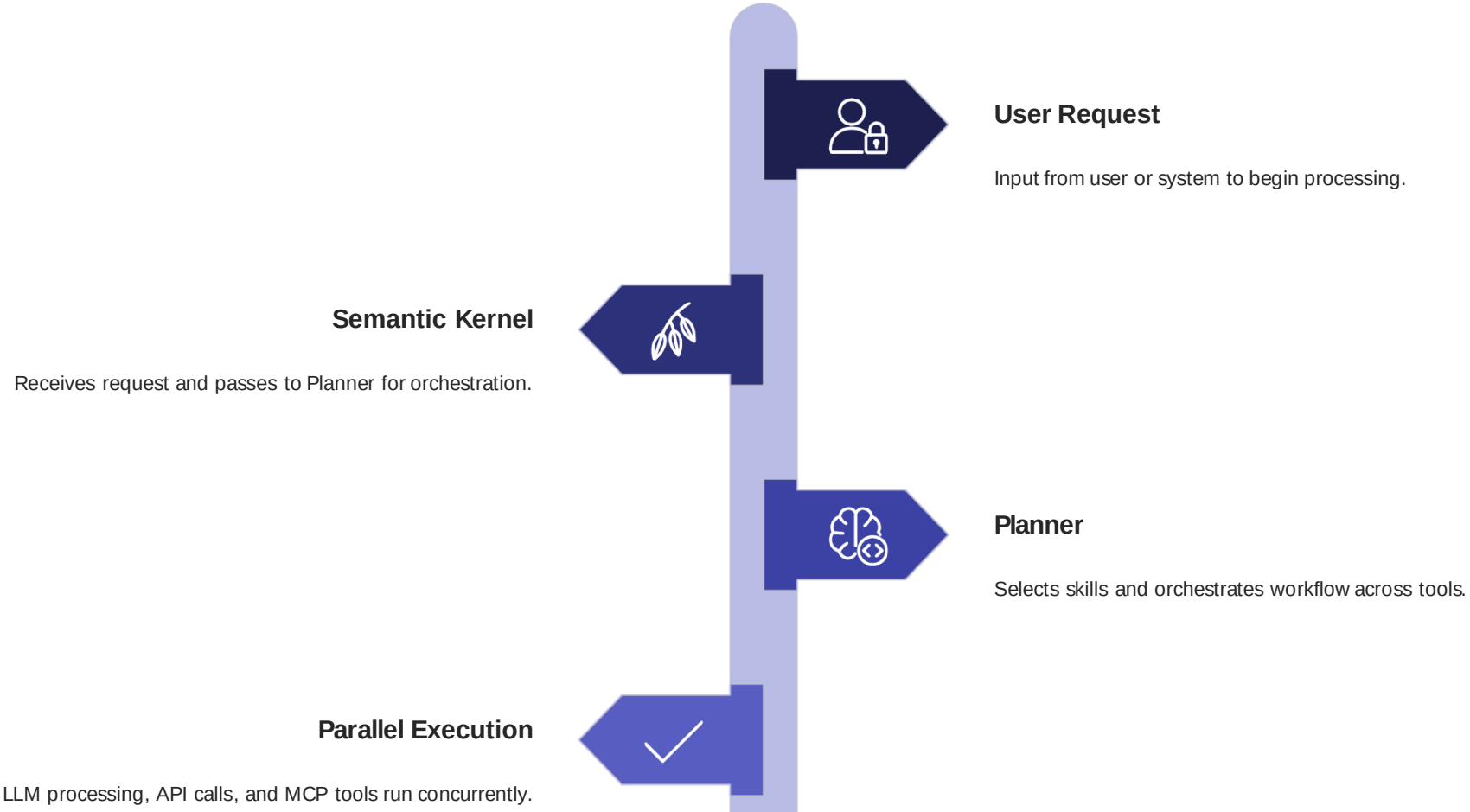
Architecture Pattern

User Request → Semantic Kernel → Planner selects skills & orchestrates workflow → LLM / APIs / MCP Tools → Result synthesized with contextual memory

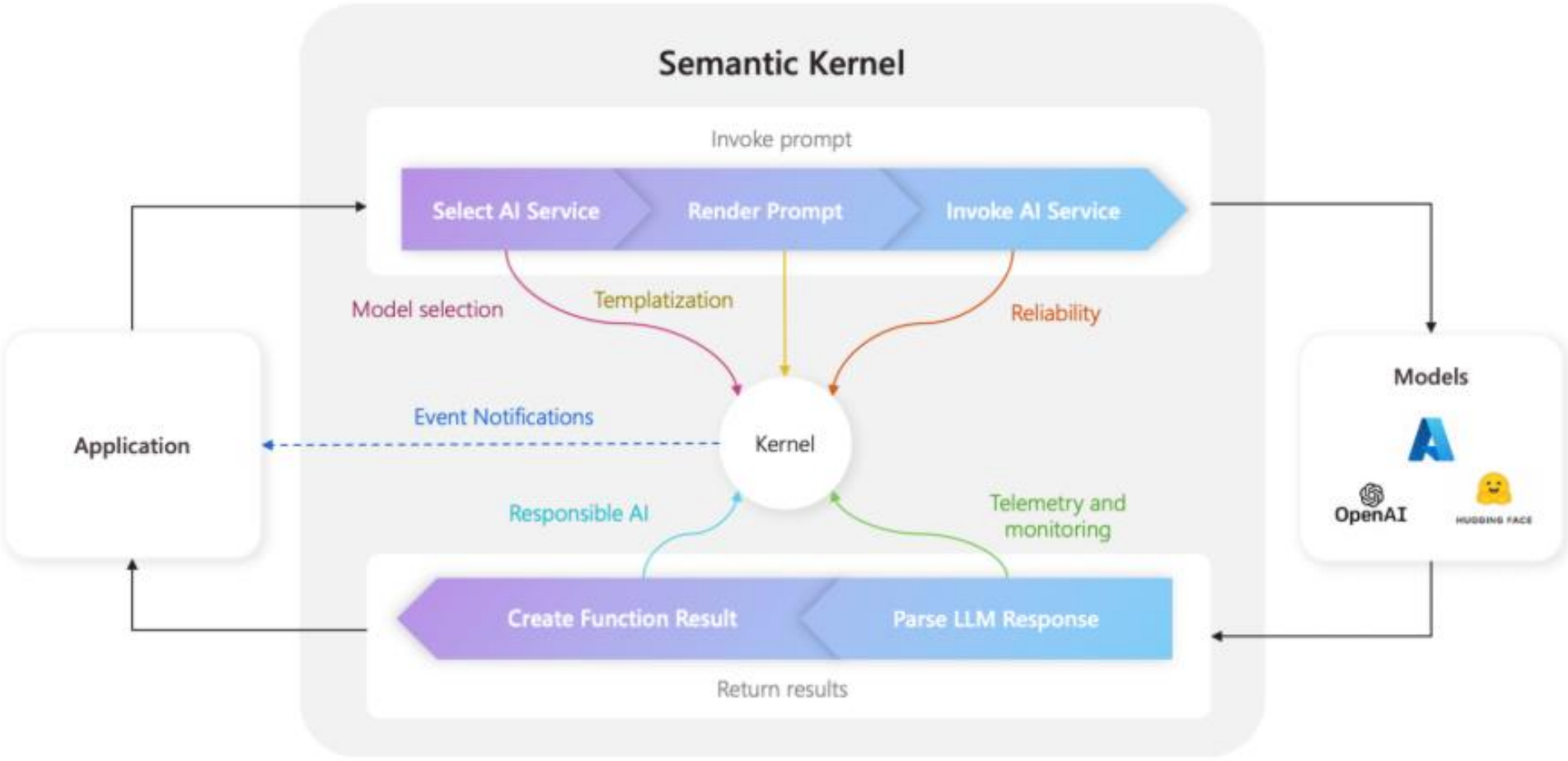
Semantic Kernel: Architecture Deep Dive

An open-source SDK that lets you *build intelligent agents with memory, skills, and orchestration* using LLMs like Azure OpenAI, GPT-4, or Mistral.

Skills Modular functions or connectors (e.g., call an API, summarize text, query SQL). Think of a "Skill" as a reusable capability that any agent can invoke dynamically.	**Planner** Dynamically decomposes user goals into smaller executable steps. Example: "Summarize GitHub issues" → Plan = fetch issues → summarize → post summary.	**Memory** Persistent, vector-based context store using embeddings for semantic recall across conversations and sessions.
Connectors Integrate with external systems — Azure services, REST APIs, MCP servers, or local tools.	**Kernel** The runtime engine that hosts LLM, skills, and planner — providing the agent "mind."	



Semantic Kernel: Architecture Deep Dive

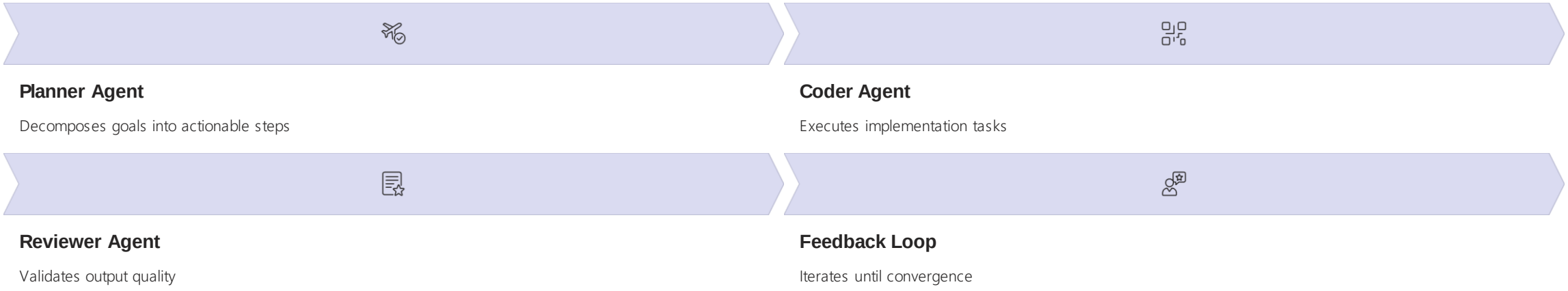


Semantic Kernel:Multi Agent Orchestration Patterns

Pattern	Description	Typical Use Case
Concurrent	Broadcasts a task to all agents, collects results independently.	Parallel analysis, independent subtasks, ensemble decision making.
Sequential	Passes the result from one agent to the next in a defined order.	Step-by-step workflows, pipelines, multi-stage processing.
Handoff	Dynamically passes control between agents based on context or rules.	Dynamic workflows, escalation, fallback, or expert handoff scenarios.
Group Chat	All agents participate in a group conversation, coordinated by a group manager.	Brainstorming, collaborative problem solving, consensus building.
Magentic	Group chat-like orchestration inspired by MagenticOne.	Complex, generalist multi-agent collaboration.

https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/agent-orchestration/?pivots=programming-language-python

AutoGen v0.4: Multi-Agent Conversation Framework



Purpose

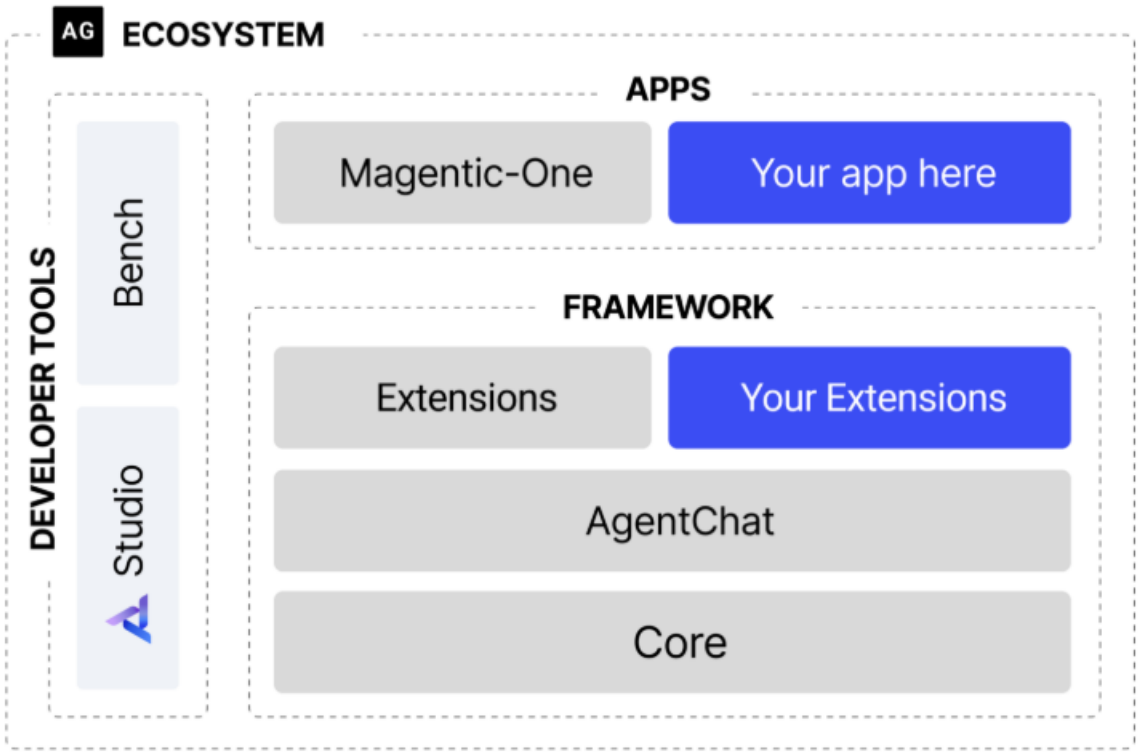
Multi-agent orchestration framework for role-based, conversational AI systems with sophisticated feedback mechanisms.

Key Features

- **Role architecture:** Specialized agents (Coder, Reviewer, Planner, Executor) with distinct capabilities and responsibilities
- Conversational task planning with context-aware feedback loops that refine outputs iteratively
- Extensible with custom agents and external tools via plugins
- Python SDK with async conversation management for high-performance scenarios
- Built-in observability and conversation debugging tools

Use Case

Multi-agent development assistants, automated code review pipelines, data analysis workflows requiring iterative refinement, or any scenario where multiple specialized agents must collaborate to solve complex problems.



https://www.microsoft.com/en-us/research/project/autogen/

Azure AI Foundry: Unified Agent Platform

Bringing It All Together: Deploying Agents on Azure Foundry

Azure AI Foundry provides a **unified platform** to create, orchestrate, and manage AI agents built with frameworks like Semantic Kernel and AutoGen — bringing enterprise-grade scalability, observability, and governance to agent deployments.

Visual & SDK-based Agent Builder

Create agents using UI or code, define skills, memory stores, and event triggers with low-code or full-code flexibility.

Multi-Agent Orchestration Support

Deploy agents that collaborate via AutoGen conversation patterns or Semantic Kernel planners with built-in coordination.

Native Azure Integrations

Connect seamlessly with Azure OpenAI, Cognitive Search, Data Explorer, Storage, and other Azure services.

Tooling & Extensions

Integrate external connectors via MCP or REST APIs for actions like GitHub operations, workflow triggers, database queries.

Secure & Managed Runtime

Built-in identity (Microsoft Entra ID), policy enforcement, cost control, and monitoring across the agent lifecycle.

portal.azure.com/#view/Microsoft_Azure_ProjectOxford/CognitiveServicesHub/~/AIFoundryPhaseTwo_overview

Microsoft Azure | Search resources, services, and docs (G+/) | Copilot | nhemamalini11@gmail.... NGGIRLS INDIA (NGGIRLSINDIA....

Home > AI Foundry

AI Foundry

ngGirls India

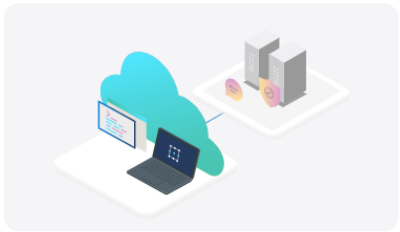
Search

- Overview
- All resources
- Use with AI Foundry
  - Azure AI Foundry
  - AI Hubs
  - Azure OpenAI
  - AI Search
- More services
  - Bot services
  - Computer vision
  - Custom vision
  - Content safety
  - Document intelligence
  - Face API
  - Health Insights
  - Machine Learning

Overview | Dashboard

Innovate Anywhere with AI

Build intelligent apps faster using prebuilt and custom models, scalable infrastructure, and tools optimized for developers and data teams.



Create an AI Foundry Resource

Start building an AI Foundry resource to start building, customizing and evaluating AI solutions using AI Foundry.

Create a resource

Manage AI Resources

View and manage all your AI services.

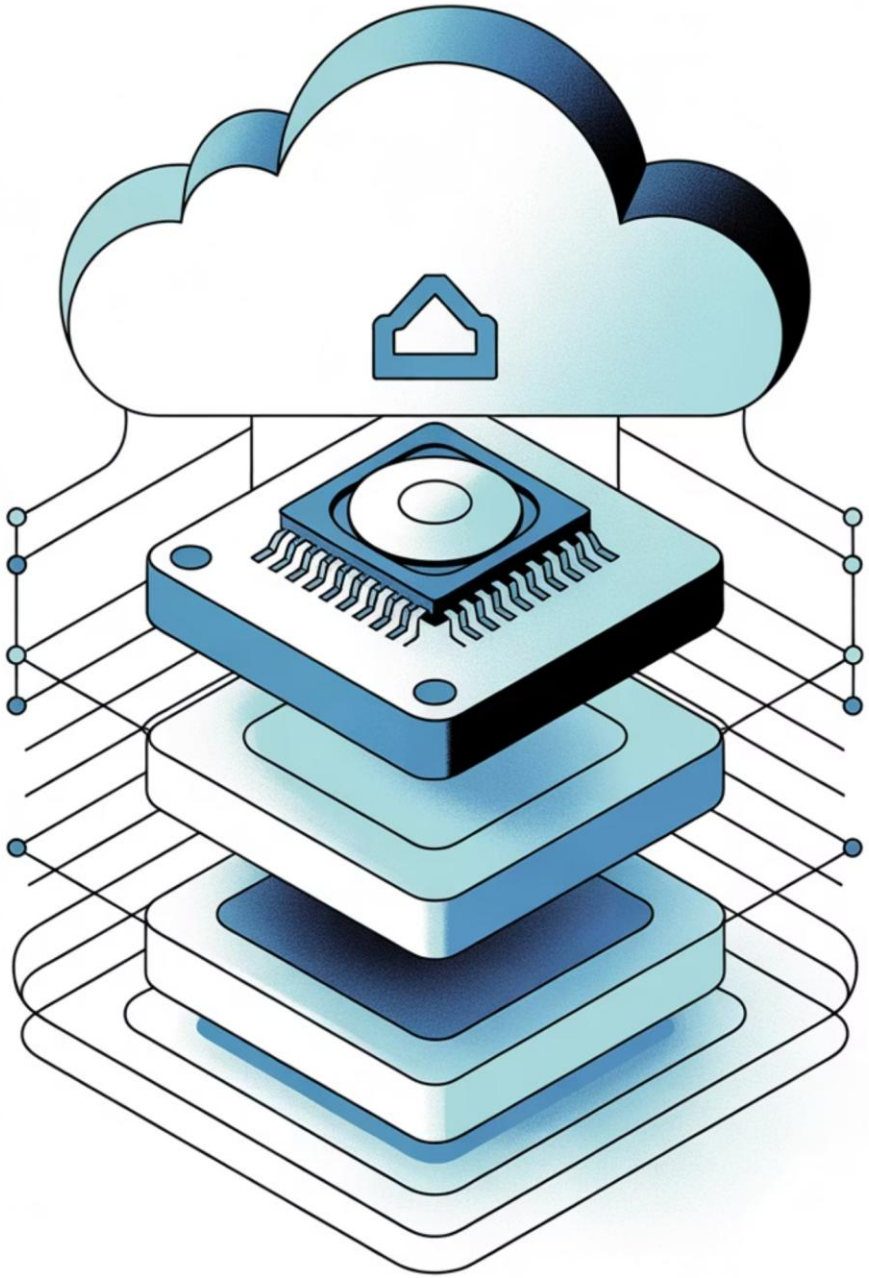
View all resources

Explore Best Practices and Guidance

Learn how to securely design, deploy, and manage AI solutions.

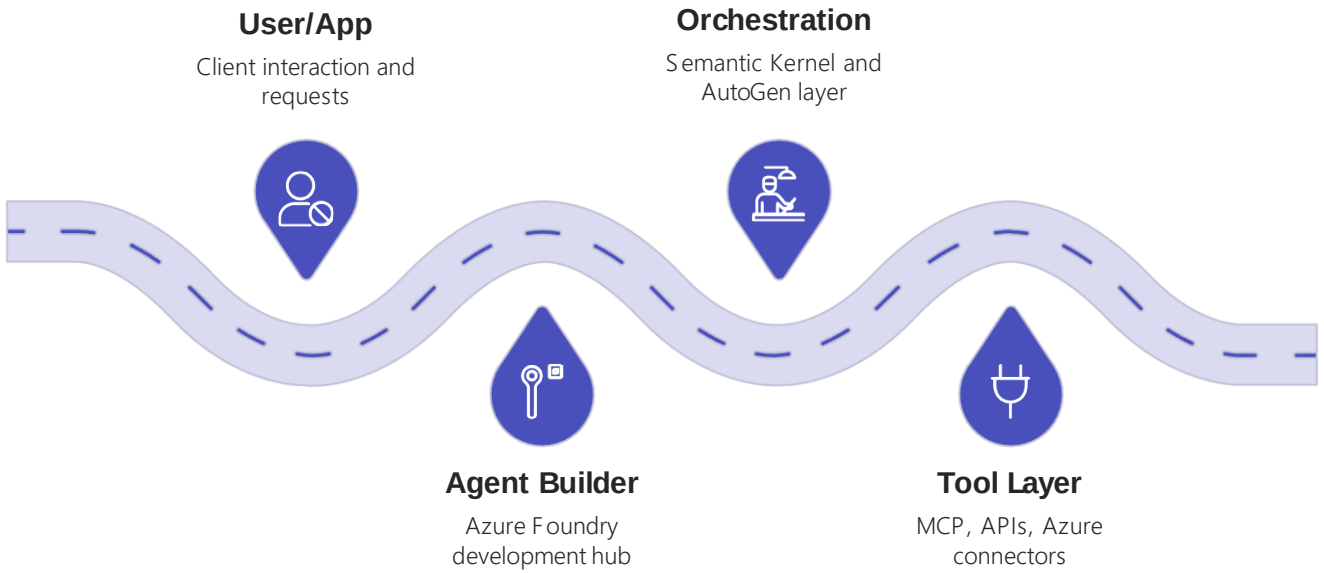
View documentation

Give feedback



Azure AI Foundry: Architecture Overview

> "Azure AI Foundry turns your agent prototype into a production-grade, governable AI service."



This layered architecture separates concerns while enabling deep integration. The Agent Builder provides development and deployment interfaces, the Orchestration Layer manages agent coordination and reasoning, the Tool Layer abstracts external integrations, and the External Systems layer represents the services agents interact with. Each layer is independently scalable and maintainable.

Microsoft Agent Framework: Unified SDK for Agents & Workflows

Microsoft Agent Framework is an open-source development kit for building both individual AI agents and multi-agent workflows, combining the strengths of Semantic Kernel and AutoGen into a single, cohesive SDK.

AI Agents

Build agents that use LLMs, call tools / MCP servers, maintain state, and process input → action → response cycles. Microsoft Learn Documentation

Workflows

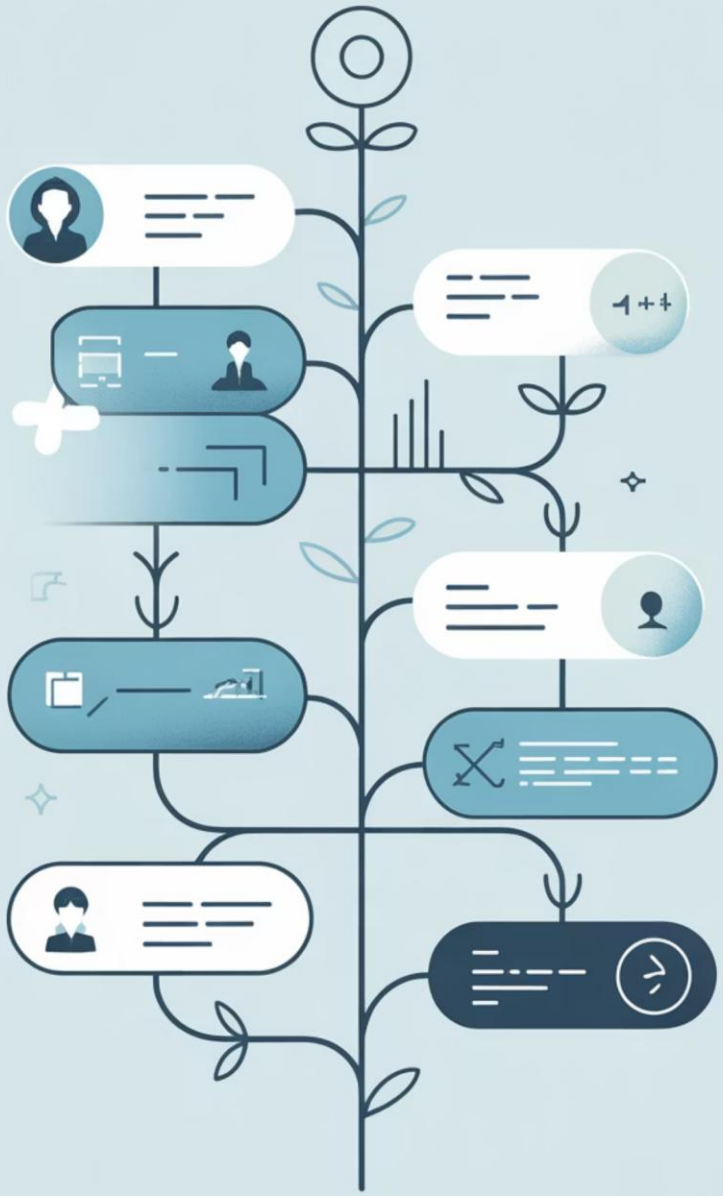
Define graph-based orchestration over agents and functions — supporting routing, nesting, human-in-the-loop, and long-running tasks. Workflow Documentation

Foundational Building Blocks

Includes model clients, agent thread/state management, context providers (memory), middleware pipeline, and MCP client integration out of the box.

Why This Framework?

It **unifies** Semantic Kernel's memory & skill abstractions with AutoGen's agent orchestration models, creating a single SDK that handles both simple single-agent scenarios and complex multi-agent workflows. Adds new workflow capabilities with **type safety**, **checkpointing**, and **conditional routing** that neither framework provided independently.



When to Use Microsoft Agent Framework

Use the Agent Framework when your solution needs:

Complex orchestration across multiple agents

When tasks require coordination between specialized agents with different capabilities, roles, and knowledge domains

Long-running, stateful processes

For workflows that span hours or days, require checkpointing, or must survive process restarts without losing context

Graph-like decision flows with human-in-loop

When approval gates, manual interventions, or conditional branching based on intermediate results are essential

Safe integration of external tool calls via MCP

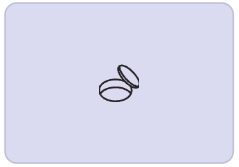
For secure, governed access to APIs, databases, and cloud services with centralized policy enforcement and audit logging

Learn more: Windows AI Studio Extension

Microsoft Agent Framework: Key Capabilities

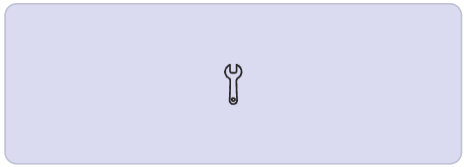
Capability	What It Means	Why It Matters
Unified SDK for .NET & Python	Developers can use same framework across languages with consistent APIs	Reduces fragmentation; easier team onboarding and skill transfer
Agent + Workflow support	Build a single agent or orchestrate multiple agents in workflow graphs	Supports both simple and complex use cases with single toolkit
Built-in MCP, OpenAPI, and A2A	Connect to external tools, cross-runtime agent calls, standardized protocols	Enables interoperability & modular extensibility without custom code
Durability, state & checkpointing	Agents can persist context, resume workflows, support long-running tasks	Critical for real-world, enterprise workloads with reliability requirements
Observability, telemetry & middleware	Intercept actions, log traces, filter or sanitize data at runtime	Helps debugging, governance, compliance, and security auditing
Convergence of AutoGen + SK	Inherits multi-agent orchestration and enterprise context management	Leverages strengths of both projects in one future-facing SDK

Agent Framework: Architectural Layers



Infrastructure & Hosting

Deployment, telemetry, state persistence — Azure AI Foundry Agents Service, Application Insights

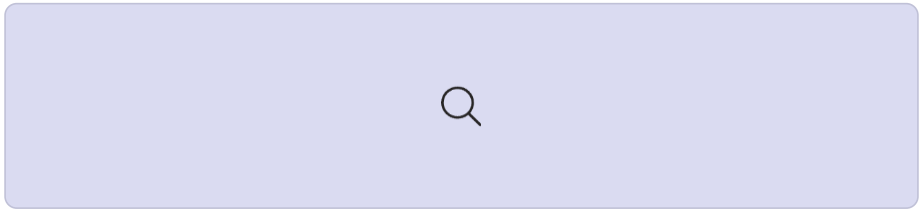


Tooling Layer

MCP servers, OpenAPI definitions, Azure connectors, custom functions — Blob Search tool, Power BI analytics

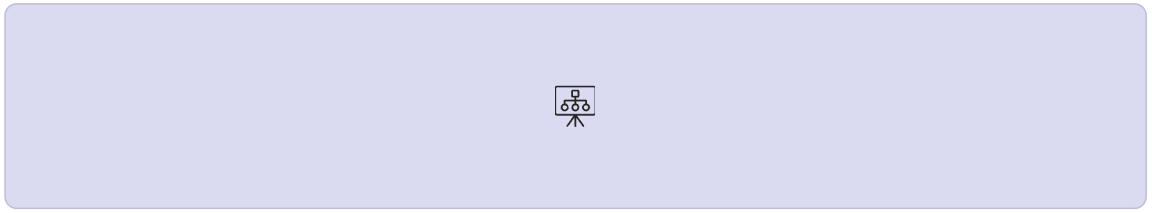
Middleware & Context

Hooks, filters, and providers for context injection — sanitize prompts, attach telemetry metadata



Agent Runtime

Core loop managing LLM, tools, memory, reasoning — Planner agent using GPT-4-Turbo with internal tools



Workflows

Graph of agents with routing, nesting, checkpointing — multi-agent pipeline orchestration

Each layer builds on the ones below it, creating a sophisticated yet maintainable architecture. Infrastructure provides the foundation, Tools enable external integration, Middleware adds cross-cutting concerns, Runtime manages agent execution, and Workflows orchestrate complex multi-agent scenarios.

Agent Framework: Core Components

Component	Description
Agent	Core class wrapping model + memory + tools + policy — the fundamental unit of intelligence
AgentContext	Carries conversation state, messages, and metadata between turns — enabling continuity
Tool	Callable external function (OpenAPI, SDK, or custom) — extends agent capabilities
Memory	Storage abstraction (vector store, CosmosDB, Redis, etc.) — enables context retention
Planner	Interprets user goals, decomposes into tool or agent calls — orchestrates execution
Policy	Defines decision-making and routing rules — controls agent behavior
Workflow	Defines graph connections between agents and tools — orchestrates multi-agent scenarios

Design Philosophy: Each component is independently testable and replaceable, following SOLID principles and enabling gradual adoption without requiring wholesale rewrites of existing agent code.

Tooling Layer: MCP, OpenAPI & Custom Tools

MCP (Model Context Protocol)

Standardized interface for context & tool exchange supporting cross-agent interoperability. Agents discover tools dynamically at runtime without hardcoded dependencies.

OpenAPI Tools

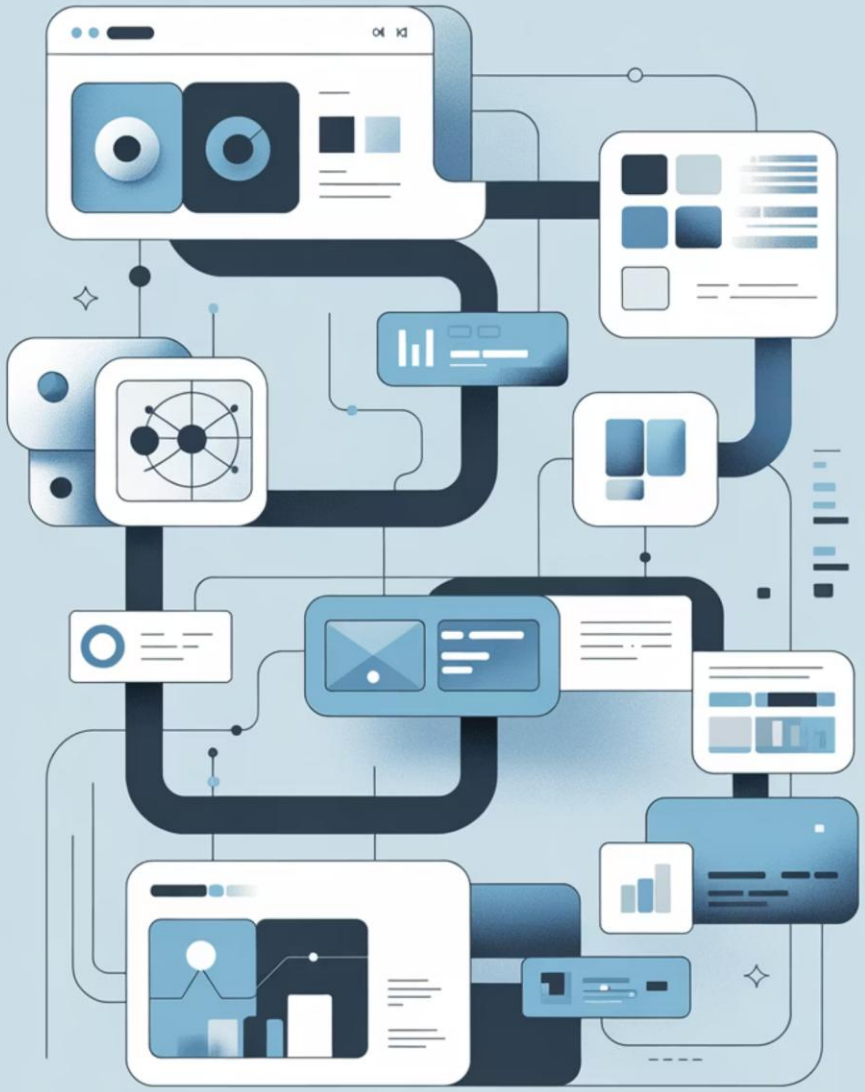
Wrap REST APIs automatically into callable tools using OpenAPI specifications. The framework generates type-safe client code from API definitions, handling authentication, serialization, and error handling.

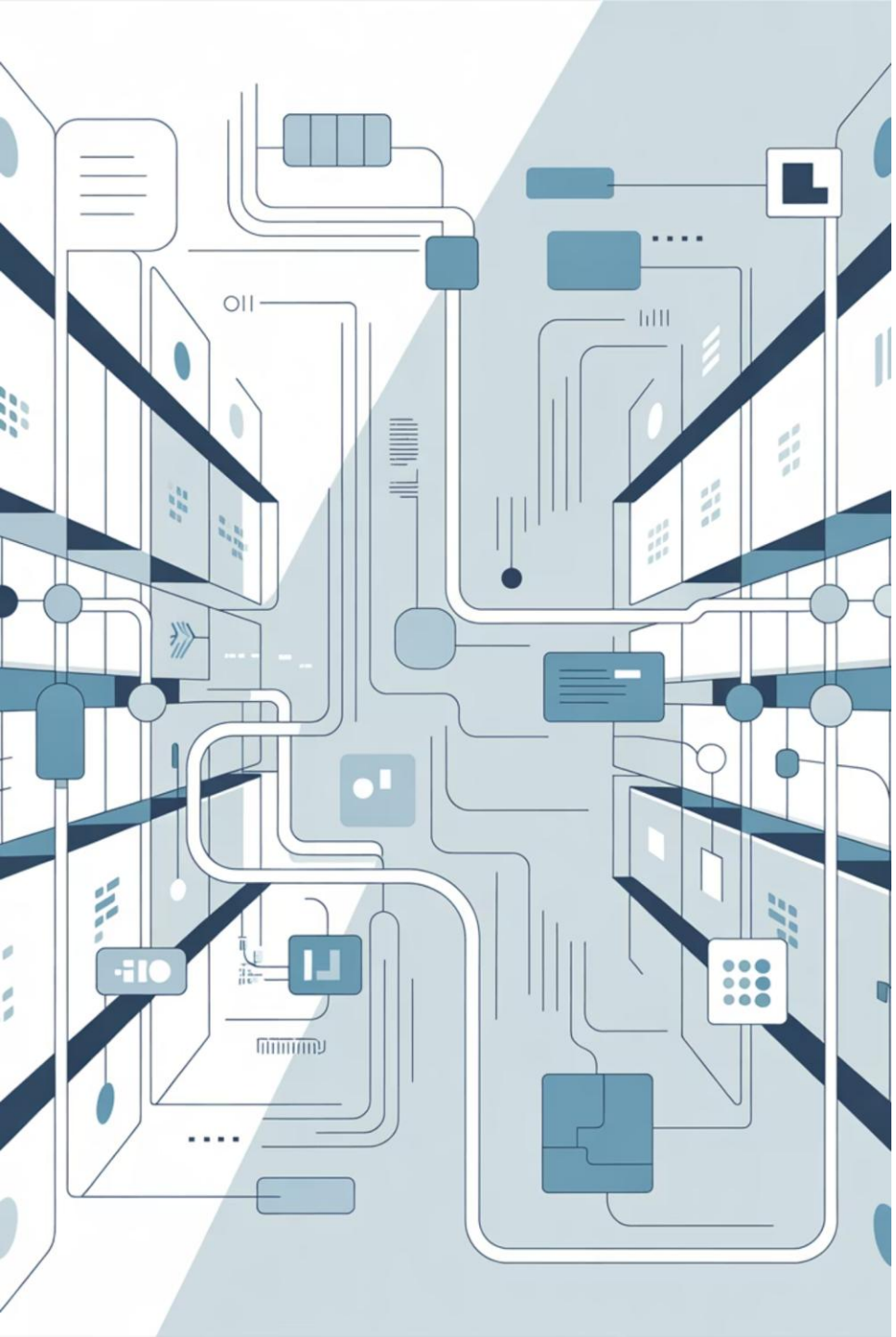
Custom Tools

Implement tool interfaces directly in .NET or Python for specialized capabilities. Custom tools have full access to the runtime context, enabling sophisticated integrations with legacy systems or proprietary APIs.

Auto-Discovery

Framework scans and registers tools automatically via decorators (Python) or attributes (.NET). Tools declare their capabilities, parameter schemas, and required permissions, enabling agent planners to reason about tool selection.





Context, Memory, and Checkpointing

01

Context Providers

Inject relevant data (user profile, conversation history, domain knowledge) into agent execution context

02

Memory Stores

Persist embeddings, summaries, or task state using vector databases, NoSQL, or blob storage

03

Checkpointing

Captures runtime state at key decision points — enabling long-running workflows, retries, and recovery

04

Resume Agent

Restore execution from checkpoint with full context preservation

Supported Stores

- **Azure Cosmos DB** — global distribution, low latency
- **Azure Blob Storage** — cost-effective persistence
- **Vector Databases** — semantic search (Azure AI Search, Pinecone)
- **Redis** — high-performance caching
- **Local disk** — development and testing

> "Agents are no longer stateless chatbots. They can pause mid-process, resume later, or share context across sessions."

Workflows and Orchestration

Multi-agent graph orchestration brings structure and reliability to complex agent interactions through declarative workflow definitions.

Declarative Definition

Define workflow graphs using YAML, JSON, or fluent API syntax — no imperative coordination code required

Nodes

Represent agents, tools, or human intervention steps — each node has inputs, outputs, and execution logic

Edges

Define routing logic between nodes — conditional branching, parallel execution, or sequential flow

Resilience Features

Built-in checkpointing, automatic retries, event-driven triggers, and error handling

Design Philosophy: "This is the AutoGen spirit reborn — a workflow graph where each agent has clear responsibility, but the orchestration logic is separated from agent implementation, enabling independent evolution."

Observability & Middleware

Telemetry Hooks

Deep integration via Application Insights and OpenTelemetry for distributed tracing

Middleware Pipeline

Pre- and post-processing of agent actions for logging, transformation, enrichment

Security Filters

Prompt sanitization, PII removal, content policy enforcement at runtime

Event Tracing

Capture reasoning chains, tool invocations, and decision points for debugging

Key Observability Capabilities

- **Distributed tracing:** Track requests across agent boundaries with correlation IDs and span tracking
- **Performance metrics:** Measure latency, token consumption, tool execution time, and success rates
- **Audit logging:** Comprehensive logs of agent actions, decisions, and data access for compliance
- **Custom instrumentation:** Inject application-specific metrics and business KPIs into telemetry stream
- **Real-time monitoring:** Dashboards and alerts for production agent health and performance

Integration with Azure AI Foundry

Local Development

Develop and test agents locally using VS Code, Visual Studio, or your preferred IDE with full .NET/Python SDK support

Agent Registration

Register agent definitions in **Azure AI Foundry (Agents Service)** via portal, CLI, or Infrastructure-as-Code

Configuration

Configure scaling policies, identity integration (Microsoft Entra ID), logging levels, and resource limits

Deployment

Deploy to Azure with automatic containerization, load balancing, and zero-downtime updates

Exposure

Expose agent endpoints via REST APIs, Copilot Studio integrations, or Teams app manifests

This seamless path from prototype to production enables rapid iteration while maintaining enterprise governance, security, and observability requirements throughout the development lifecycle.

Hands-On Demo: Building Your First Agent

we'll dive into the practical application of the Microsoft Agent Framework by building a simple yet engaging agent together. Get ready to turn theory into practice as we craft a pirate joke agent using Python.

This demo is designed to be interactive and illustrative, giving you a clear pathway to start developing your own intelligent agents. By the end of this session, you will learn:

1. **Setting Up Your Environment**

   We'll guide you through the necessary steps to prepare your development environment for agent creation, ensuring you have all the tools you need.

2. **Creating an Agent with Custom Instructions**

   Discover how to define an agent's persona and behavior using custom instructions, bringing our pirate joke teller to life.

3. **Running Conversations**

   Learn to initiate and manage conversations with your newly built agent, observing its responses and capabilities in action.

https://github.com/microsoft/agent-framework/tree/main/python

Demo Prerequisites & Setup

To follow along with our hands-on demo and build your own pirate joke agent, please ensure you have the following prerequisites and setup steps completed:

1

Python 3.10+

Ensure you have Python version 3.10 or higher installed on your development machine. This version is required for full compatibility with the Microsoft Agent Framework SDK.

```
python --version
```

2

Azure AI Project & Deployed Model

You'll need an active Azure AI project with a deployed language model, specifically `gpt-4o-mini`. This will serve as the underlying large language model for our agent.

- Create an Azure AI project if you don't have one.
- Deploy the `gpt-4o-mini` model within your project.

3

Azure CLI Installation & Authentication

Install the Azure Command-Line Interface (CLI) to manage your Azure resources. Authenticate your CLI session to access your Azure subscription.

```
az login
```

Follow the browser prompts to log in to your Azure account.

4

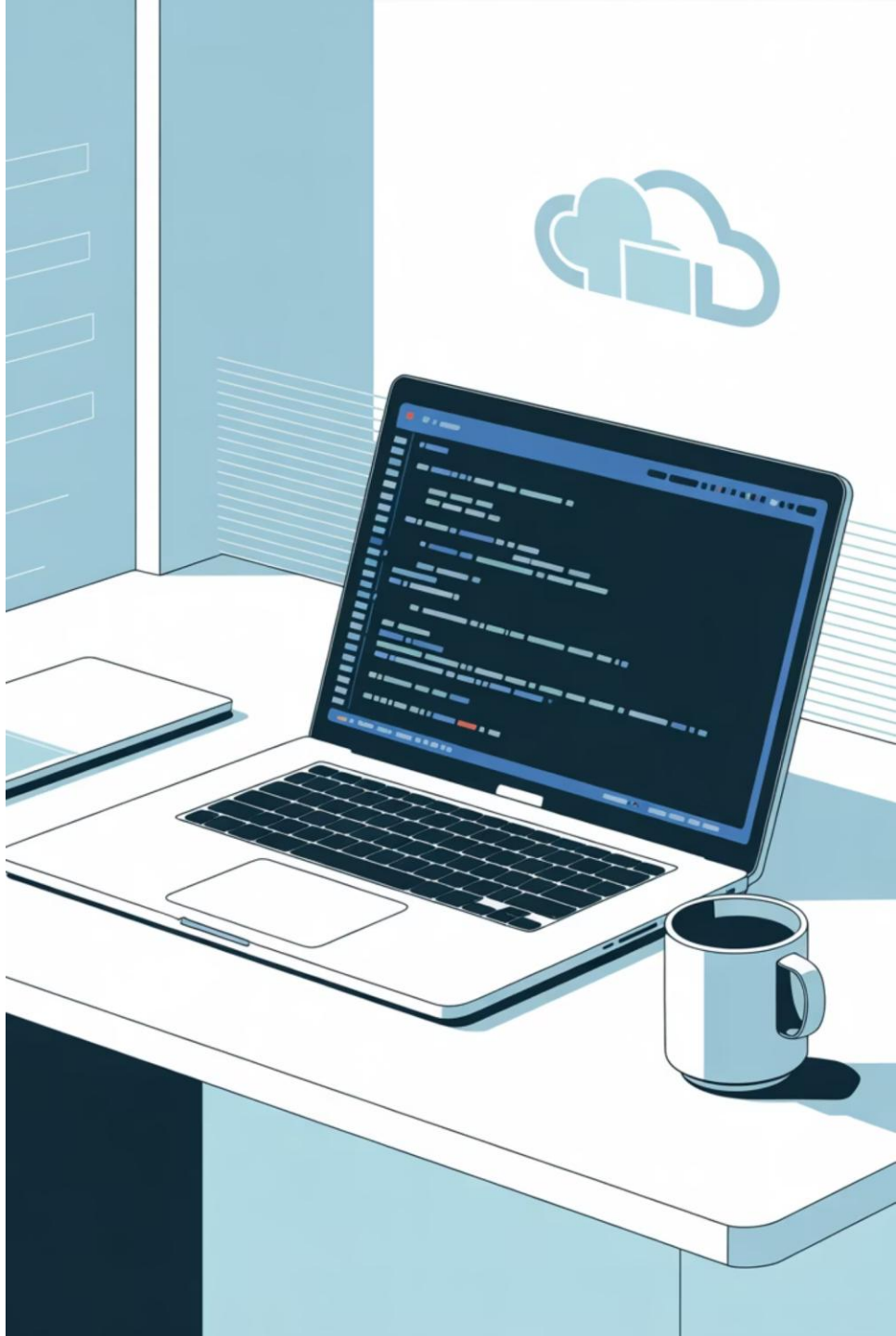
Environment Variables Setup

Configure your environment variables to point to your Azure AI project endpoint and deployed model name. This allows the agent framework to connect to your Azure AI resources.

```
export AZURE_AI_PROJECT_ENDPOINT="your_azure_ai_project_endpoint"export AZURE_AI_MODEL_DEPLOYMENT_NAME="gpt-4o-mini"
```

Replace `"your_azure_ai_project_endpoint"` with the actual endpoint from your Azure AI project settings.

Having these elements in place will ensure a smooth and successful demo experience. Let's get started!



Python Agent Code Walkthrough

```
#Creating the agent

import asyncio
from agent_framework.azure import AzureOpenAIChatClient
from azure.identity import AzureCliCredential

agent = AzureOpenAIChatClient(credential=AzureCliCredential()).create_agent(
    instructions="You are good at telling jokes.",
    name="Joker"
)

#Running the agent
async def main():
    result = await agent.run("Tell me a joke about a pirate.")
    print(result.text)

asyncio.run(main())

#Running the  agent with streamin
async def main():
    async for update in agent.run_stream("Tell me a joke about a pirate."):
        if update.text:
            print(update.text, end="", flush=True)
    print()  # New line after streaming is complete

asyncio.run(main())
```

Running the Demo

Let's run the Python script we just walked through and see our pirate joke agent in action!

Execute the Agent

```
python your_agent_script.py
```

Expected Output

```
User: Tell me a joke about a pirate.Agent: Why did the pirate go to art school? Because he wanted to learn how to drawrrrrr!
```

Behind the Scenes: What's Happening?

1

1. Script Execution

Your Python script initializes the `AzureAIAgentClient` and `ChatAgent`.

2

2. Azure AI Connection

The client uses your Azure CLI credentials to connect securely to your Azure AI project endpoint.

3

3. LLM Processing

The agent forwards your prompt "Tell me a joke about a pirate." to the deployed `gpt-4o-mini` model, adhering to its custom instructions.

4

4. Response Generation

The LLM generates a pirate joke based on its training and your agent's instructions, which is then returned and printed by your script.

Troubleshooting Tips

Authentication Issues

Ensure you're logged into Azure CLI with `az login` and have the correct permissions for your Azure AI project. Verify your credentials are active.

Environment Variables

Double-check that `AZURE_AI_PROJECT_ENDPOINT` and `AZURE_AI_MODEL_DEPLOYMENT_NAME` are correctly set and accessible in your shell. Restart your terminal if necessary after setting them.

Model Deployment

Confirm that the `gpt-4o-mini` model is indeed deployed within your Azure AI project and the deployment name matches exactly.

Extending the Demo: Next Steps

The basic pirate joke agent is a great starting point, but the Microsoft Agent Framework allows for much more sophisticated interactions. Let's explore how you can extend its capabilities and build truly intelligent and versatile agents for real-world applications. The possibilities are vast, and with these next steps, you can transform a simple demo into a powerful, intelligent system.

→ Adding conversation memory

Enable your agent to remember past interactions and maintain context throughout a conversation, leading to more natural and coherent dialogue.

→ Integrating tools/MCP servers

Equip your agent with the ability to perform actions, access external data, or interact with other systems, expanding its functional reach beyond mere conversation.

→ Creating multi-agent workflows

Design complex systems where multiple agents collaborate, each specializing in a task, to achieve a larger, more intricate goal.

→ Deploying to Azure AI Foundry

Scale and manage your agents in a robust production environment, leveraging Azure's infrastructure for reliability and performance.

Adding Memory to Your Agent

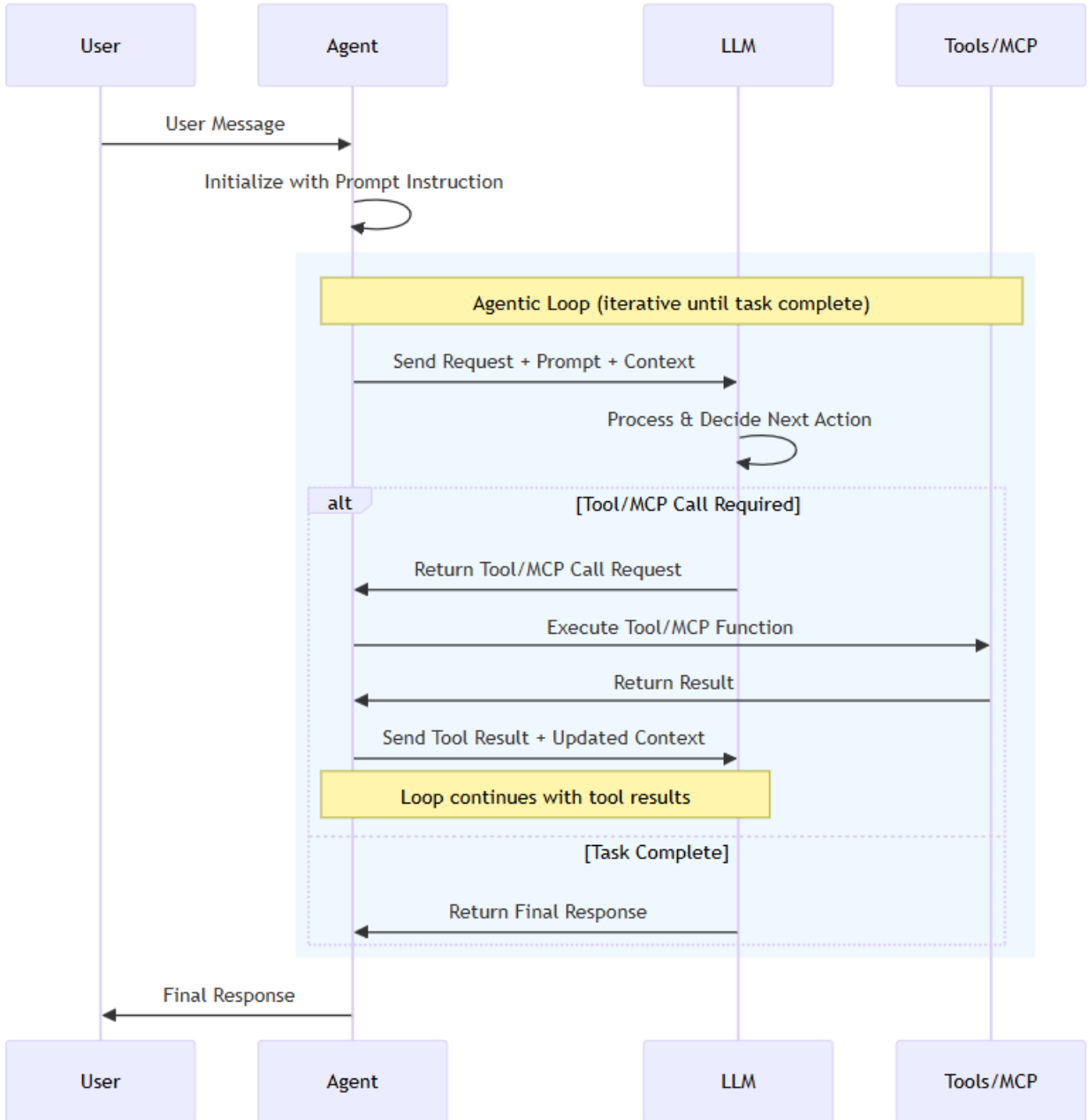
Enhance your agent's ability to hold contextual conversations by integrating a memory component. The framework provides built-in mechanisms to manage conversation history.

Integrating Tools with Your Agent

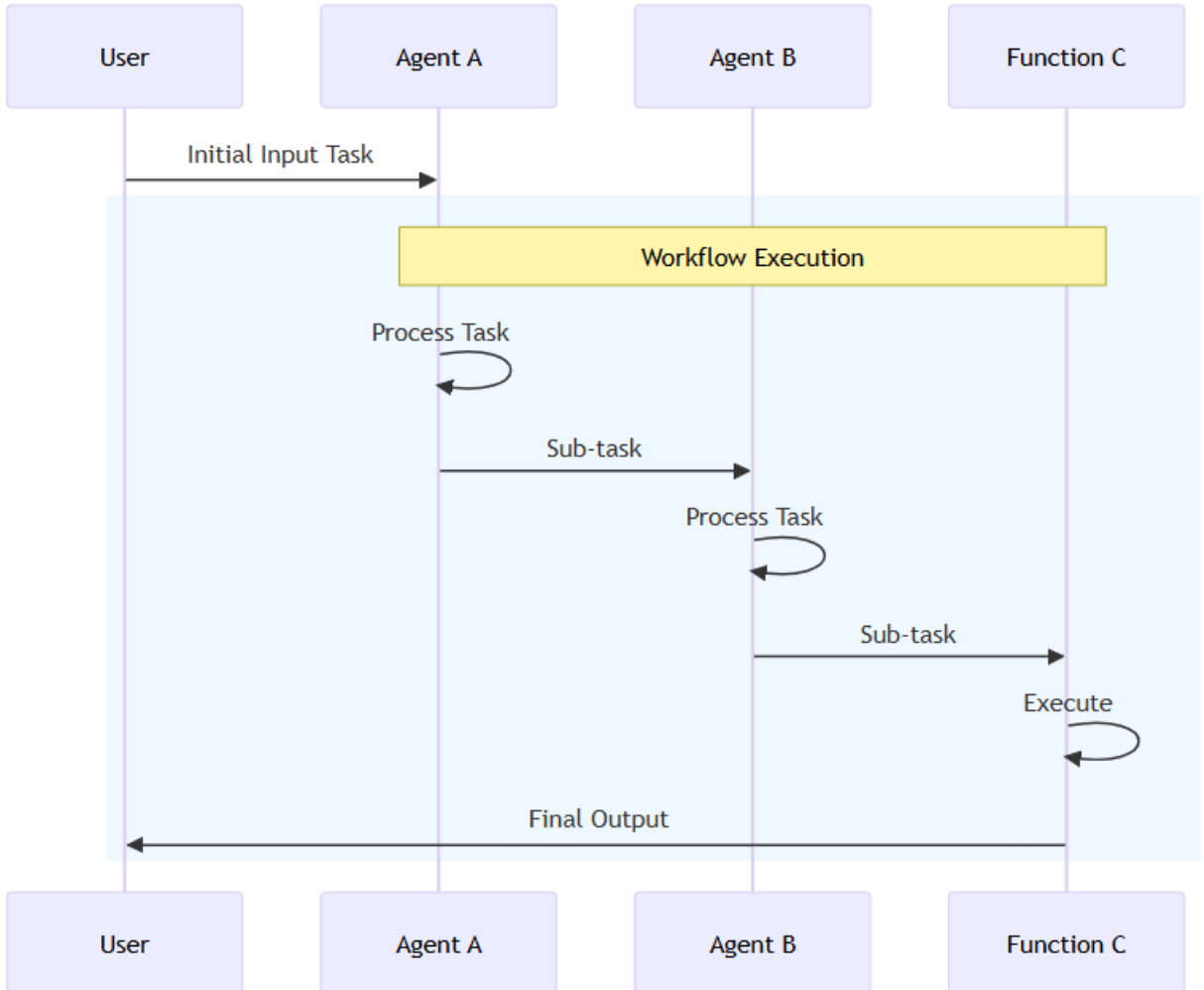
Python Copy

```python
from typing import Annotated
from pydantic import Field

def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
) -> str:
    """Get the weather for a given location."""
    return f"The weather in {location} is cloudy with a high of 15°C."
```

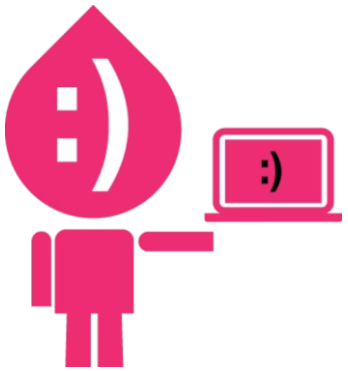


https://learn.microsoft.com/en-us/agent-framework/overview/agent-framework-overview



https://learn.microsoft.com/en-us/agent-framework/tutorials/workflows/simple-sequential-workflow?pivots=programming-language-python

Thank You — Let's Build the Future Together



Q&A

Questions? Let's discuss how to architect your MCP-enabled agent solutions on Azure.

Resources

- GitHub Repository →
- Agent Framework Quick Start →
- Connect on LinkedIn →

TQRCG